COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S1
DATE PERFORMED:	04/08/2026	**DATE SUBMITTED:**	11/08/2026
NAME:	Al Twain D. Pasion	**INSTRUCTOR:**	Engr Jimlord Quejado

ACTIVITY NO. 5 Queues

1. PROCEDURE OUTPUT

What procedures did you perform?

ILOA

CODE

```cpp
#include <iostream>
#include <queue>

void display(std::queue<char> r);
void empty(std::queue<char> o);
void size (std::queue<char> b);
void front(std::queue<char> l);
void back(std::queue<char> o);

int main(){

    std::queue<char> q;

    q.push('c');
    q.push('p');
    q.push('e');
    q.push('0');
    q.push('1');
    q.push('0');

    //display
    display(q);
    //empty
    empty(q);
    //size
    size(q);
    //front
    front(q);
    //back
    back(q);
    //pop
    q.pop();
    display(q);
    //push another elemet
    q.push('e');
    display(q);

    //delete all
    while (!q.empty()){
        q.pop();
    }
    display(q);


    return 0;
}
```

```
47
48  void display(std::queue<char> r){
49
50      std::queue<char> c = r;
51
52      if (c.empty()){
53          std::cout << "The queue is empty\n";
54          return;
55      }
56
57      while(!c.empty()){
58          std::cout << c.front() << " ";
59          c.pop();
60      }
61      std::cout << "\n";
62  }
63
64  void empty(std::queue<char> r){
65      if (r.empty()){
66          std::cout << "The queue is empty\n";
67      }
68      else {
69          std::cout << "The queue is not empty\n";
70      }
71  }
72
73  void size(std::queue<char> b){
74      std::cout << "There are " << b.size() << " things in line\n";
75  }
76
77  void front(std::queue<char> l){
78      std::cout << l.front() << " is at the front\n";
79  }
80
81  void back(std::queue<char> o){
82      std::cout << o.back() << " is at the back\n";
83  }
84
```

OUTPUT

```
c p e 0 1 0
The queue is not empty
There are 6 things in line
c is at the front
0 is at the back
p e 0 1 0
p e 0 1 0 e
The queue is empty
PS C:\Users\TIPQC\Downloads\temp\HOA 5\TLOA>
```

ANALYSIS:

In this code, I tried making each individual function for the queue stl functions. The display makes a temporary queue identical to the original queue and prints out the front before popping it until it is empty. The empty() function returns a boolean value so it will check if it's empty or not. The size() function returns an integer value of the size of the queue. Lastly, both front() and back() prints out the data at the front and back of the queue. The delete method was a bit straightforward, I just popped the queue until it was empty.

ILOB1

CODE

```cpp
#ifndef QUEUELL_H
#define QUEUELL_H

template <typename T>
class qNode{
    public:
    T data;
    qNode* next;
};


//create new node
template <typename T>
qNode<T>* new_node(T newData){
    qNode<T>* newNode = new qNode<T>;
    newNode->data = newData;
    newNode->next = nullptr;
    return newNode;
}


template <typename T>
void enqueue(qNode<T>** frontPtr, qNode<T>** backPtr, T newData){
    qNode<T>* newNode = new_node(newData);

    if ((*frontPtr) == nullptr && (*backPtr) == nullptr){
        (*frontPtr) = newNode;
        (*backPtr) = newNode;
    }

    (*backPtr)->next = newNode;
    (*backPtr) = newNode;
}

template <typename T>
void dequeue(qNode<T>** frontPtr, qNode<T>** backPtr){
    qNode<T>* deleteNode = nullptr;
    deleteNode = (*frontPtr);

    if ((*frontPtr)->next == nullptr && (*backPtr)->next == nullptr){
        (*frontPtr) = nullptr;
        (*backPtr) = nullptr;
        delete deleteNode;
        return;
    }

    (*frontPtr) = (*frontPtr)->next;
    deleteNode->next = nullptr;
    delete deleteNode;

}
```

```cpp
template <typename T>
void display(qNode<T>** frontPtr, qNode<T>** backPtr){
    qNode<T>* temp = (*frontPtr);

    if ((*frontPtr)->next == nullptr && (*backPtr)->next == nullptr){
        std::cout << "Queue is empty\n";
        return;
    }

    do {
        std::cout << temp->data << " ";
        temp = temp->next;

    } while(temp != nullptr);
    std::cout << "\n";
}

template <typename T>
T Front(qNode<T>** frontPtr){
    return (*frontPtr)->data;
}

#endif
```

```cpp
#include <iostream>
#include "queuell.h"

int main(){

    qNode<char>* front = nullptr;
    qNode<char>* back = nullptr;

    enqueue(&front, &back, 'T');
    enqueue(&front, &back, 'W');
    enqueue(&front, &back, 'A');
    enqueue(&front, &back, 'I');
    enqueue(&front, &back, 'N');
    display(&front, &back);
    dequeue(&front, &back);
    display(&front, &back);

    std::cout << Front(&front) << "\n";

    return 0;
}

```

OUTPUT

```
T W A I N
W A I N
W
PS C:\Users\TIPQC\Downloads\temp\HOA 5\ILOB>
```

ANALYSIS:

For this activity, we made a queue header file that uses linked lists. The functions have different purposes like, creating a new node and putting it at the back of the line, removing the node at the front of the line, displaying all the data in the queue starting from the front, and only displaying the data at the front. It has similar properties and concepts as of a linked list, so it is easy to understand and modify. The only rule to follow is the FIFO, or "First In, First Out" rule. This states that the first data entered will always be the first data out, and the latest data entered will be put at the very back. This concept applies to deleting and putting new data to the queue.

ILOB2

CODE

```cpp
#ifndef SUP_ACT_5_QUEUEARRAY_H
#define SUP_ACT_5_QUEUEARRAY_H

template <typename T>
class qarray {
private:
    T* q_array = nullptr;
    size_t q_capacity = 8;
    size_t q_size = 0;
    int q_front = -1;
    int q_back = -1;

public:
    qarray(size_t capacity);
    bool Empty();
    bool Full();

    size_t Size();
    void Clear();

    T Front();
    T Back();

    void Enqueue(T newElement);
    void Dequeue();

    ~qarray();
    qarray(const qarray& other);
    qarray& operator=(const qarray& other);
};

template <typename T>
qarray<T>::qarray(size_t capacity) {
    q_capacity = capacity;
    q_size = 0;
    q_front = 0;
    q_back = -1;
    q_array = new T[q_capacity];
}

template <typename T>
qarray<T>::~qarray() {
    delete [] q_array;;
}

```

```cpp
template <typename T>
qarray<T>::qarray(const qarray& other) {
    q_capacity = other.q_capacity;
    q_size = other.q_size;
    q_front = other.q_front;
    q_back = other.q_back;
    q_array = new T[q_capacity];
    for (size_t i = 0; i < q_capacity; i++) {
        q_array[i] = other.q_array[i];
    }
}

template <typename T>
qarray<T>& qarray<T>::operator=(const qarray& other) {
    if (this == &other) {
        return *this;
    }
    delete [] q_array;

    q_capacity = other.q_capacity;
    q_size = other.q_size;
    q_front = other.q_front;
    q_back = other.q_back;
    q_array = new T[q_capacity];
    for (size_t i = 0; i < q_capacity; i++) {
        q_array[i] = other.q_array[i];
    }
    return *this;
}

template <typename T>
size_t qarray<T>::Size() {
    return q_size;
}

template <typename T>
bool qarray<T>::Empty() {
    return q_size == 0;
}

template <typename T>
bool qarray<T>::Full() {
    return q_size == q_capacity;
}
```

```cpp
template <typename T>
void qarray<T>::Clear() {
    q_size = 0;
    q_front = -1;
    q_back = -1;
}

template <typename T>
T qarray<T>::Front() {
    if (Empty()) {
        std::cout<<"Q is empty\n";
        return T{};
    }
    return q_array[q_front];
}

template <typename T>
T qarray<T>::Back() {
    if (Empty()) {
        std::cout<<"Q is empty\n";
        return T{};
    }
    return q_array[q_back];
}

template <typename T>
void qarray<T>::Enqueue(T newElement) {
    if (Full()) {
        std::cout<<"Q is full\n";
        return;
    }

    q_back = (q_back + 1) % q_capacity;
    q_array[q_back] = newElement;
    q_size++;
}

template <typename T>
void qarray<T>::Dequeue() {
    if (Empty()) {
        std::cout<<"Q is empty\n";
        return;
    }

    q_front = (q_front + 1) % q_capacity;
    q_size--;
}

#endif //SUP_ACT_5_QUEUEARRAY_H
```

```cpp
#include <iostream>
#include "queuearray.h"

int main() {

    qarray<> q( capacity: 5);
    std::cout << "q_empty: " << q.Empty() << " q_full: " << q.Full() << " q_size: " << q.Size() << std::endl;

    q.Enqueue( newElement: 1);
    q.Enqueue( newElement: 2);
    q.Enqueue( newElement: 3);
    q.Enqueue( newElement: 4);
    q.Enqueue( newElement: 5);
    std::cout << "New Elements -> q_front: " << q.Front() << " q_back: " << q.Back() << " q_empty: " << q.Empty() << " q_full: " << q.Full() << " q_size: " << q.Size() << std::endl;

    q.Enqueue( newElement: 6);

    q.Dequeue();
    q.Dequeue();
    std::cout << "After 2 dequeues -> q_front: " << q.Front() << " q_back: " << q.Back() << " q_size: " << q.Size() << std::endl;

    q.Enqueue( newElement: 6);
    q.Enqueue( newElement: 7); // wraps around to the start of the array
    std::cout << "After wraparound -> q_front: " << q.Front() << " q_back: " << q.Back() << " q_size: " << q.Size() << std::endl;

    qarray<> Copy( & q);
    Copy.Dequeue();
    Copy.Enqueue( newElement: 99);
    std::cout << "q_back: " << q.Back() << " Copy_back: " << Copy.Back() << std::endl;

    qarray<> Assign( capacity: 3);
    Assign = q;
    std::cout << "Assign_front: " << Assign.Front() << " Assign_back: " << Assign.Back() << " qAssign_size: " << Assign.Size() << std::endl;

    q.Clear();
    std::cout << "After Clear -> q_empty: " << q.Empty() << " q_size: " << q.Size() << std::endl;

    q.Dequeue();
    return 0;
}
```

OUTPUT

```
q_empty: 1 q_full: 0 q_size: 0
New Elements -> q_front: 1 q_back: 5 q_empty: 0 q_full: 1 q_size: 5
Q is full
After 2 dequeues -> q_front: 3 q_back: 5 q_size: 3
After wraparound -> q_front: 3 q_back: 7 q_size: 5
q_back: 7 Copy_back: 99
Assign_front: 3 Assign_back: 7 qAssign_size: 5
After Clear -> q_empty: 1 q_size: 0
Q is empty
```

ANALYSIS:

This activity now implements a queue, using an array. Our logic here is that the array would be circular, meaning we won't be wasting processing time by shifting the elements to the very front every dequeue. For the main driver function, we first called the constructor by declaring a new array with the capacity of 5. We then queued five elements in the array and showed that it's full by queuing an extra element. After that, we dequeue 2 elements and queued 2 more to demonstrate the wraparound or "circular" behavior of the array. Here is when we called the copy constructor, we declared a new object with the existing array, and then queued a new element to the copied constructor, lastly we demonstrated the copy assignment operator by declaring a new array and assigning it as the previous array. We then cleared the front and back and checked for emptiness.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Solve problems using queue implementation

Problem Title: Shared Printer Simulation using Queues
Problem Definition: In this activity, we'll simulate a queue for a shared printer in an office. In any corporate office, usually, the printer is shared across the whole floor in the printer room. All the computers in this room are connected
to the same printer. But a printer can do only one printing job at any point in time, and it also takes some time to complete any job. In the meantime, some other user can send another print request. In such a case, a printer needs

to store all the pending jobs somewhere so that it can take them up once its current task is done.
Perform the following steps to solve the activity. Make sure that you include a screenshot of the source code for each. From the get-go: You must choose whether you are making a linked list or array implementation.
You may NOT use the STL.

1. Create a class called Job (comprising an ID for the job, the name of the user who submitted it, and the number of pages).
2. Create a class called Printer. This will provide an interface to add new jobs and process all the jobs added so far.
3. To implement the printer class, it will need to store all the pending jobs. We'll implement a very basic strategy – first come, first served. Whoever submits the job first will be the first to get the job done.
4. Finally, simulate a scenario where multiple people are adding jobs to the printer, and the printer is processing them one by one.
5. Defend your choice of internal representation: Why did you use arrays or linked lists?

Output Analysis:
- Provide the output after performing each task above.
- Include your analysis: focus on why you think the output is the way it is.

CODE

```cpp
#ifndef HOA_5_SUP5_H
#define HOA_5_SUP5_H

template <typename T>
class Printer {
public:
    T data;
    Printer* next;
};

// create new node
template <typename T>
Printer<T>* new_node(T newData) {
    Printer<T>* newNode = new Printer<T>;
    newNode->data = newData;
    newNode->next = nullptr;
    return newNode;
}

template <typename T>
void enqueue(Printer<T>** frontPtr, Printer<T>** backPtr, T newData) {
    Printer<T>* newNode = new_node(newData);

    if ((*frontPtr) == nullptr && (*backPtr) == nullptr) {
        (*frontPtr) = newNode;
        (*backPtr) = newNode;
        return;
    }

    (*backPtr)->next = newNode;
    (*backPtr) = newNode;
}

template <typename T>
void dequeue(Printer<T>** frontPtr, Printer<T>** backPtr) {
    if ((*frontPtr) == nullptr) {
        std::cout << "Queue is empty\n";
        return;
    }

    Printer<T>* deleteNode = (*frontPtr);

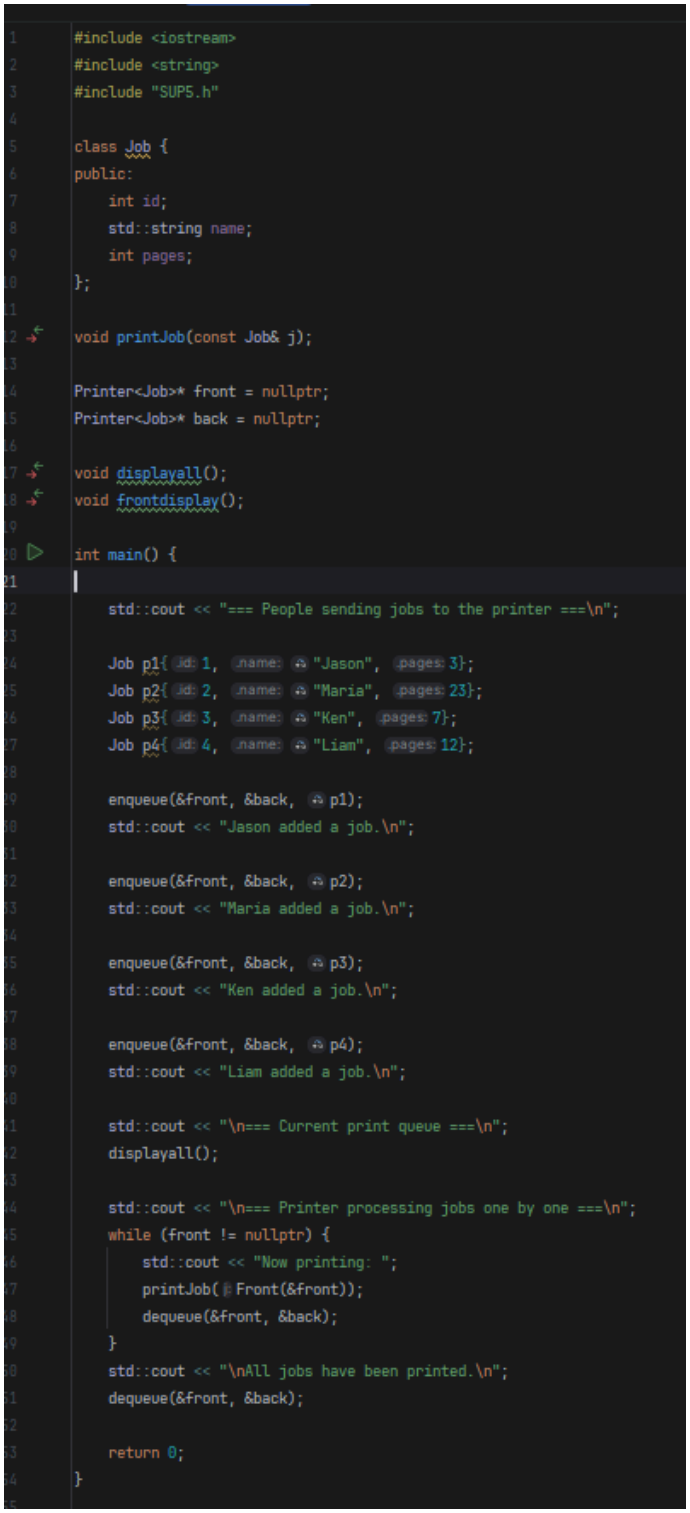
    if ((*frontPtr) == (*backPtr)) {

        (*frontPtr) = nullptr;
        (*backPtr) = nullptr;
        delete deleteNode;
        return;
    }

    (*frontPtr) = (*frontPtr)->next;
    deleteNode->next = nullptr;
    delete deleteNode;
}
```

```
55
56  template <typename T>
57  T Front(Printer<T>** frontPtr) {
58      return (*frontPtr)->data;
59  }
60
61  #endif //HOA_5_SUP5_H
```

main driver function



```cpp
#include <iostream>
#include <string>
#include "SUP5.h"

class Job {
public:
    int id;
    std::string name;
    int pages;
};

void printJob(const Job& j);

Printer<Job>* front = nullptr;
Printer<Job>* back = nullptr;

void displayall();
void frontdisplay();

int main() {

    std::cout << "=== People sending jobs to the printer ===\n";

    Job p1{ id: 1,  name:  "Jason",  pages: 3};
    Job p2{ id: 2,  name:  "Maria",  pages: 23};
    Job p3{ id: 3,  name:  "Ken",  pages: 7};
    Job p4{ id: 4,  name:  "Liam",  pages: 12};

    enqueue(&front, &back,  p1);
    std::cout << "Jason added a job.\n";

    enqueue(&front, &back,  p2);
    std::cout << "Maria added a job.\n";

    enqueue(&front, &back,  p3);
    std::cout << "Ken added a job.\n";

    enqueue(&front, &back,  p4);
    std::cout << "Liam added a job.\n";

    std::cout << "\n=== Current print queue ===\n";
    displayall();

    std::cout << "\n=== Printer processing jobs one by one ===\n";
    while (front != nullptr) {
        std::cout << "Now printing: ";
        printJob( Front(&front));
        dequeue(&front, &back);
    }
    std::cout << "\nAll jobs have been printed.\n";
    dequeue(&front, &back);

    return 0;
}
```

```cpp
void printJob(const Job& j) {
    std::cout << "ID: " << j.id << " Name: " << j.name << " Pages: " << j.pages << "\n";
}

void displayall() {
    Printer<Job>* temp = front;
    while (temp != nullptr) {
        printJob(temp->data);
        temp = temp->next;
    }
}

void frontdisplay() {
    printJob(Front(&front));
}
```

OUTPUT

```
=== People sending jobs to the printer ===
Jason added a job.
Maria added a job.
Ken added a job.
Liam added a job.

=== Current print queue ===
ID: 1 Name: Jason Pages: 3
ID: 2 Name: Maria Pages: 23
ID: 3 Name: Ken Pages: 7
ID: 4 Name: Liam Pages: 12

=== Printer processing jobs one by one ===
Now printing: ID: 1 Name: Jason Pages: 3
Now printing: ID: 2 Name: Maria Pages: 23
Now printing: ID: 3 Name: Ken Pages: 7
Now printing: ID: 4 Name: Liam Pages: 12

All jobs have been printed.
Queue is empty
```

ANALYSIS & DEFENCE:

I choose to use linked-lists because it is dynamic, there is no max limit. The program flow is like this, 4 people added jobs and the printer displays all the current jobs in-queue. After that, it will “print” all the jobs, FIFO. In this case, it will print in this sequence, Jason, Maria, Ken, and lastly Liam. Now it will display that everything has been printed and that the queue is finally empty.

CONCLUSION
Queue is a bit like “two sides of the same coin” relationship with Stack. One follows LIFO, while the other one follows FIFO. Implementing them using linked-lists and arrays was a bit challenging to understand. Personally, I had a hard time understanding arrays. The best way to visualize a stack is to imagine a tissue dispenser. You can only get the top and insert at the top, you technically need to open up the back part to access all of it but the concept is there. For queues, it’s like a one-way tunnel whoever goes in first will always come out first all the time. It’s a very challenging concept to understand, but what worked best for me was to relate it to real-life concepts.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Explanation of Concepts**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Explaining the flow and process of the code.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.’s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.